

SIMATS ENGINEERING

ASSESSMENT TOOL 3 – MOCK INTERVIEW QUESTIONS

COURSE CODE: ITA 0606 | COURSE NAME: MACHINE LEARNING

5 QUESTIONS | 20 MARKS

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)

Assessment Tool Weightage: 10%

MOCK INTERVIEW QUESTIONS

Code of Conduct:

I ADHITHYA.S 192412352 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Hospital Patient Data – Near Real-Time Cleaning

Question

You are working as a data analyst in a hospital where patient data is continuously collected and stored in CSV files. However, the dataset contains missing values, inconsistent units, and duplicate entries. The system must process this data in near real-time for reporting. Explain how you would design a solution using NumPy and Pandas to clean, filter, and standardize the data efficiently. Discuss indexing, aggregation, and saving the processed data, along with strategies to handle unexpected data anomalies.

Interview Answer

Answer – Opening

I would build a streaming-style batch pipeline using Pandas for tabular cleaning and NumPy for efficient numerical operations. Each incoming CSV is validated, cleaned, standardized and appended to a processed dataset.

1. Load and Validate

I would use `pandas.read_csv()` and first check shape, data types, missing values, duplicate patient IDs and required columns. Unexpected files would be logged instead of crashing the pipeline.

2. Clean and Standardize

Missing numerical values can be handled using an appropriate imputation method such as median. Invalid values are detected using boolean filtering. Units such as temperature or weight must be converted to one standard unit before analysis. Categorical values such as Male/male/M are mapped to one standard representation.

3. Indexing and Filtering

I would use `.loc` for label-based selection and boolean masks to select invalid or clinically impossible records. A meaningful `Patient_ID` or timestamp can be used for indexing, but the index should not replace the actual patient identifier.

4. Aggregation

Pandas `groupby()` can calculate daily patient counts, average temperature, admission counts or other reporting statistics. NumPy aggregation functions can efficiently process numerical arrays.

5. Saving

After validation, I would save the cleaned data using `to_csv()` or another suitable format. For near-real-time processing, I would process only newly arrived files/records rather than repeatedly processing the entire historical dataset.

6. Anomalies and Reliability

I would maintain an anomaly log for corrupted rows, unexpected units, impossible values and duplicate IDs. I would use `try/except` for file errors and assertions or validation checks for important constraints.

Conclusion

This approach gives a fast and auditable pipeline. Pandas handles structured cleaning and grouping, while NumPy provides efficient numerical processing. The key is to validate every incoming batch before it reaches the reporting layer.

Quick Interview Version

Load and validate each incoming batch → clean missing/invalid values → standardize units/categories → filter anomalies → aggregate for reports → save validated data.

Key Terms to Remember

`read_csv()`, `isna()`, `fillna()`, `drop_duplicates()`, `loc`, boolean filtering, `groupby()`, aggregation, `to_csv()`, vectorization.

2. Financial Transactions – Large-Scale Analysis

Question

A financial company provides you with millions of transaction records that must be analyzed to detect unusual spending patterns. Due to system limitations, memory and computation time are critical constraints. Describe how you would use NumPy arrays and Pandas DataFrames to perform filtering, grouping, and statistical analysis efficiently. Explain how you would optimize indexing and aggregation operations, and how you would debug performance bottlenecks or incorrect outputs.

Interview Answer

Answer – Opening

For millions of transactions, I would design a memory-efficient vectorized pipeline. Pandas would manage tabular transaction data, while NumPy would perform numerical calculations efficiently.

1. Efficient Loading

I would read only required columns using `usecols` and specify appropriate dtypes. For files larger than available memory, I would use `chunksize` and process one chunk at a time.

2. Filtering

Vectorized boolean conditions can identify high-value transactions, unusual frequencies or transactions outside expected ranges. This is faster than iterating row by row in Python.

3. Grouping and Statistics

`groupby()` can calculate total spending, average transaction value and transaction counts per customer. NumPy functions such as `mean()`, `median()` and `std()` can be used for numerical statistics.

4. Indexing Optimization

An index can be useful when repeated lookups are performed on a stable key such as `Customer_ID` or `timestamp`. I would avoid creating unnecessary indexes because they consume memory.

5. Detecting Unusual Patterns

I could calculate customer-level statistics and flag transactions that are far from normal values, for example using a threshold based on mean and standard deviation or another suitable rule.

6. Debugging Performance

I would check memory usage, DataFrame dtypes, number of rows and intermediate object sizes. I would time individual pipeline stages to find bottlenecks and avoid unnecessary copies, loops and repeated groupby operations.

7. Debugging Incorrect Results

I would inspect a small sample manually, check duplicate transactions, missing values, data types and aggregation definitions. I would compare results against a small known dataset.

Conclusion

Vectorization, compact dtypes, chunk processing, early filtering and careful aggregation allow millions of records to be processed within memory and time constraints.

Quick Interview Version

Use compact dtypes and chunking → vectorized NumPy/Pandas filtering → groupby aggregation → optimized indexing → profile memory/time → validate results with small known samples.

Key Terms to Remember

NumPy arrays, dtype, nbytes, vectorization, chunksize, boolean masks, groupby(), aggregation, profiling.

3. Combining Departmental Datasets

Question

You are given multiple datasets from different departments (sales, marketing, and customer support), each stored in different file formats. Your task is to combine these datasets into a unified DataFrame while ensuring no loss of critical information and maintaining consistent indexing. Explain your approach using Pandas for file I/O, combining, grouping, and sorting. How would you validate the merged dataset and resolve conflicts such as missing or duplicate keys?

Interview Answer

Answer – Opening

I would first standardize each source separately and then combine them using a clearly defined common key, such as Customer_ID. I would preserve unmatched records when the requirement is to avoid loss of critical information.

1. File I/O

I would use `read_csv()` for CSV files and the appropriate Pandas reader for other supported formats. Each file would be checked for required columns, data types and encoding/format problems.

2. Standardization

Before merging, I would standardize Customer_ID data types, remove unnecessary spaces and make column names consistent. This prevents false mismatches.

3. Combining

I would use `merge()` for relational datasets sharing a common key. An outer join is appropriate when records from both sources must be preserved; left or inner joins are selected according to the reporting requirement.

4. Duplicate and Missing Keys

I would use `duplicated()` and `groupby().size()` to identify duplicate keys. Missing keys would be reported and handled according to the business rule rather than silently discarded.

5. Indexing

After merging, I would sort by the common key and use `reset_index(drop=True)` to create a clean sequential DataFrame index. The Customer_ID remains the actual business key.

6. Validation

I would compare record counts before and after merging, check duplicate keys, inspect missing values and use `merge(..., indicator=True)` to identify matched and unmatched records.

7. Grouping and Sorting

`groupby()` can summarize sales, campaign responses or support cases by customer. `sort_values()` can order the final report by customer, date or a calculated metric.

Conclusion

The merge is considered correct only after key consistency, completeness, duplicate handling and record-count checks pass. This prevents silent data loss.

Quick Interview Version

Read and standardize each source → align common keys/types → merge with the correct join → detect duplicates/unmatched records → group/sort → validate counts and completeness.

Key Terms to Remember

`read_csv()/other readers`, `merge()`, `concat()`, outer join, `duplicated()`, `groupby()`, `sort_values()`, `reset_index()`, merge indicator.

4. Automated Daily Data Analysis Pipeline

Question

A company requires an automated system that reads daily data files, processes them, and generates summary reports along with visualizations. The pipeline must filter relevant data, compute statistics, and produce sorted outputs for dashboards. Describe how you would design this workflow using NumPy and Pandas, including file handling, filtering, aggregation, and plotting. Also explain how you would handle errors such as corrupted files, inconsistent formats, or missing columns.

Interview Answer

Answer – Opening

I would design the workflow as a sequence: file discovery → validation → loading → cleaning → filtering → aggregation → sorting → report generation → plotting.

1. File Handling

The system identifies new files for the current day and attempts to read each one. File-reading operations are placed inside exception handling so a corrupted file is logged without stopping the complete pipeline.

2. Schema Validation

Before processing, I would verify that required columns exist and that important columns have the expected data types. Inconsistent column names can be mapped to a standard schema.

3. Cleaning and Filtering

Missing and invalid values are handled according to predefined rules. Boolean indexing filters only records relevant to the dashboard, such as a selected date range, region or value threshold.

4. Aggregation

Pandas `groupby()` with `sum()`, `mean()`, `count()` or another suitable aggregation creates the summary table. NumPy can be used for numerical statistics.

5. Sorting

The summary is sorted using `sort_values()` so the dashboard receives a predictable and meaningful order.

6. Plotting

Matplotlib or Pandas plotting can generate bar charts, line charts or other suitable visualizations. Titles, axis labels and units should be included.

7. Error Handling

Corrupted files, missing columns, malformed numbers and unexpected categories should be logged with the file name and error reason. Invalid records should be isolated rather than silently changing the complete dataset.

Conclusion

A modular pipeline makes the process reliable and easy to debug. Each stage validates its output before passing data to the next stage.

Quick Interview Version

Discover files → validate schema → load → clean/filter → group and aggregate → sort → generate plots/reports → log errors without stopping the whole pipeline.

Key Terms to Remember

File discovery, exception handling, schema validation, filtering, groupby(), aggregation, sorting, Matplotlib/Pandas plotting.

5. E-Commerce Clickstream Preprocessing

Question

You are working on an e-commerce platform where customer behavior data is collected in real time. The dataset contains clickstream data with missing values and inconsistent formats. Explain how you would preprocess and structure this data using Pandas. Discuss filtering, feature extraction, indexing strategies, and how you would prepare the data for downstream machine learning tasks.

Interview Answer

Answer – Opening

I would convert raw clickstream events into a clean event-level DataFrame and then create customer/session-level features suitable for machine learning.

1. Structure the Data

Typical fields include Customer_ID, Session_ID, Timestamp, Page, Event_Type, Product_ID and Duration. I would standardize column names and data types immediately after loading each batch.

2. Handle Missing and Inconsistent Data

I would identify missing values with `isna()`. For essential identifiers such as Customer_ID or Timestamp, records may need to be rejected or separately flagged. Non-critical numerical fields can be imputed when appropriate. Timestamp formats should be converted using `to_datetime()`.

3. Filtering

Boolean indexing can remove invalid events, impossible durations and unwanted bot/test traffic. For example, retain events where duration is non-negative and Event_Type belongs to an allowed set.

4. Feature Extraction

Useful features include session duration, number of clicks, pages viewed, product-category counts, average time per page, cart additions and purchase indicators. These can be created using `groupby()`, aggregation and time-based operations.

5. Indexing

Timestamp can be converted to datetime and used for time-based analysis. A MultiIndex involving Session_ID and Timestamp may be useful for ordered event analysis, but I would avoid unnecessary indexes in a high-volume real-time system.

6. ML Preparation

After cleaning, categorical variables can be encoded, numerical variables scaled where required, and features separated from the target variable. Data leakage must be avoided by ensuring future events do not enter features used to predict earlier outcomes.

7. Real-Time Considerations

Process new event batches incrementally, avoid repeatedly rebuilding the entire historical DataFrame, and validate schema and data quality for every incoming batch.

Conclusion

The result is a structured, validated feature table that preserves useful customer behavior while removing inconsistent events. Pandas is especially useful for timestamp handling, filtering, grouping and feature engineering before machine learning.